

中原大學
資訊工程學系
碩士學位論文

記憶體內計算之字線激活和ADC位元精
度的分層調整方法

Layer-wise Adjustment Method for Word-line Activation and ADC Bit Precision of CIMs

指導教授：鄭維凱

研 究 生：張嘉軒

中 華 民 國 113 年 7 月

摘要

隨著 CNN(Convolutional Neural Network) 在機器學習、圖像識別和其他人工智能領域的應用日益廣泛，大量數據的計算與傳輸需求對傳統計算架構造成了極大的壓力。傳統的馮·諾伊曼架構由於其記憶體和處理器之間頻繁數據傳輸，面臨著記憶體牆(Memory Wall)的挑戰。為了解決這一瓶頸，CIM(Compute-In-Memory)架構被提出，成為一種具有潛力的解決方案。CIM 架構通過在記憶體中進行計算，有效減少了數據在處理器和記憶體之間的傳輸，從而提升了性能。

在大多數的 CIM 架構中，Word-Line 的激活數量和 ADC(Analog-to-digital converter) 的位元精度通常是固定的。然而，這種固定配置在不同層的 CNN 計算中存在一定的局限性。CNN 的不同層在計算需求和特性上有所不同，固定的 Word-Line 激活數量和 ADC 位元精度可能導致某些層的計算過剩，能量浪費，或者計算不足，導致結果不準確。因此，如何在保持 CNN 高準確率的同時，提高系統的靈活性和效率便成為了一個重要課題。

針對上述問題，本論文提出了一種逐層調整 Word-Line 激活數量和 ADC 位元精度的方法，以適應 CNN 不同層的計算需求。透過動態調整 Word-Line 的激活數量和 ADC 的位元精度，在保障 CNN 準確率的同時，做到降低系統的延遲和能耗的效果。

ABSTRACT

As CNN (Convolutional Neural Network) is increasingly used in machine learning, image recognition and other artificial intelligence fields, the computing and transmission requirements of large amounts of data have put great pressure on traditional computing architectures. The traditional von Neumann architecture faces the challenge of the Memory Wall due to frequent data transfers between the memory and the processor. To solve this bottleneck, the CIM (Compute-In-Memory) architecture was proposed as a potential solution. The CIM architecture effectively reduces data transmission between the processor and memory by performing calculations in memory, thus improving energy efficiency and performance.

In most CIM architectures, the activation number of Word-Line and the bit accuracy of ADC (Analog-to-digital converter) are usually fixed. However, this fixed configuration has certain limitations in CNN calculations at different layers. Different layers of CNN have different computing requirements and characteristics. The fixed number of Word-Line activations and the number of ADC bits may lead to over-calculation, waste of energy, or under-calculation in some layers, resulting in inaccurate results. Therefore, how to improve the flexibility and efficiency of the system while maintaining the high accuracy of CNN has become an important topic.

In response to the above problems, this paper proposes a method to adjust the number of Word-Line activations and the number of ADC bits layer by layer to adapt to the computing needs of different layers of CNN. By dynamically adjusting the number of Word-Line activations and the number of ADC bits, we can ensure the accuracy of CNN while reducing system latency and energy consumption.

目錄

摘要	I
ABSTRACT	II
目錄	III
圖目錄	IV
表目錄	V
第一章、緒論	1
第二章、背景研究	4
2.1 ADC 位元精度對準確率的影響	4
2.2 ADC 位元精度對能耗的影響	6
2.3 電流偏差與 Word-Line 激活數量問題	9
2.4 整合 ADC 位元精度和 Word-Line 激活數量策略	11
第三章、研究方法	12
3.1 研究動機與問題闡述	12
3.1.1 ADC 位元精度	12
3.1.2 字線激活數量	16
3.2 CIM 模擬工具	18
3.3 逐層考量字線激活數量與 ADC 位元精度決策方法	20
第四章、實驗結果	24
4.1 實驗設置	24
4.2 實驗結果	32
第五章、結論	37
參考文獻	38

圖目錄

圖一、CIM 架構[6].....	1
圖二、在 CIM 架構中的 ADC[7].....	2
圖三、ADC 位元精度與量化誤差關係	4
圖四、ADC 位元精度與動態能耗關係	6
圖五、ADC 位元精度與靜態能耗關係	7
圖六、CIM 架構下的電流偏差	9
圖七、AWL 決策方法[6]	10
圖八、不同 ADC 位元精度在每一層的能耗比較	12
圖九、不同位元精度的準確率比較圖	14
圖十、AWL 對準確率的影響	16
圖十一、AWL 對每一層的個別影響	17
圖十二、模擬工具架構	18



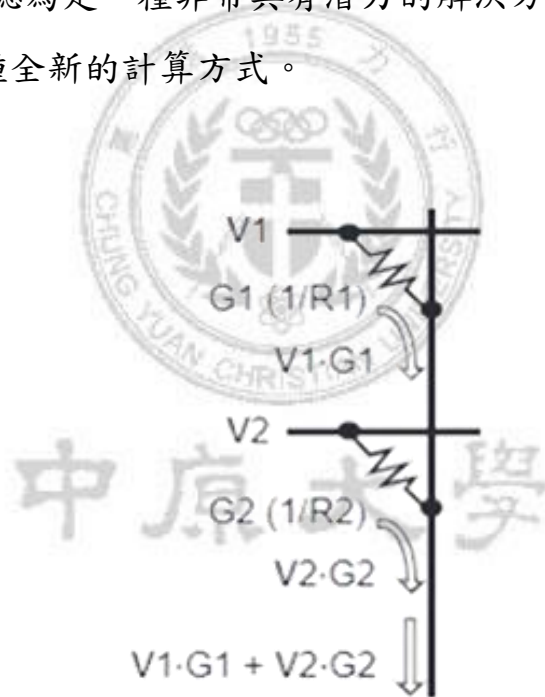
中原大學

表目錄

表格一、不同位元精度 ADC 在各層的能耗表現	13
表格二、不同位元精度 ADC 的延遲	13
表格三、不同位元精度 ADC 的準確率	15
表格四、AWL 的風險指數評估策略[13]	20
表格五、AWL 的配置對照表[13]	21
表格六、ADC 的風險評估策略	22
表格七、ADC 的配置對照表	22
表格八、考量 AWL 狀態下 ADC 的風險評估策略	23
表格九、實驗配置	24
表格十、Cifar-10 架構	25
表格十一、Cifar-10 中 ADC 的風險因子	25
表格十二、Cifar-10 中 ADC 的風險指數與配置	26
表格十三、Cifar-10 中 AWL 的風險因子	26
表格十四、Cifar-10 中 AWL 的風險指數與配置	27
表格十五、VGG-8 架構	27
表格十六、VGG-8 中 ADC 的風險因子	28
表格十七、VGG-8 中 ADC 的風險指數與配置	28
表格十八、VGG-8 中 AWL 的風險因子	29
表格十九、VGG-8 中 AWL 的風險指數與配置	29
表格二十、Cifar-10 中考量 AWL 下 ADC 的風險因子	30
表格二十一、Cifar-10 中考量 AWL 下 ADC 的風險指數與配置	30
表格二十二、VGG-8 中考量 AWL 下 ADC 的風險因子	31
表格二十三、VGG-8 中考量 AWL 下 ADC 的風險指數與配置	31
表格二十四、Cifar-10 中固定 ADC 與逐層配置 ADC 準確率	32
表格二十五、Cifar-10 中固定 ADC 與逐層配置 ADC Latency	32
表格二十六、Cifar-10 固定 AWL 與逐層配置 AWL 的 Cycle 數比較	33
表格二十七、VGG-8 中固定 ADC 與逐層配置 ADC 準確率	33
表格二十八、VGG-8 固定 ADC 與逐層配置 ADC Latency	34
表格二十九、VGG-8 固定 AWL 與逐層配置 AWL 的 Cycle 數比較	34
表格三十、考量動態配置 AWL 下 Cifar-10 中 ADC 準確率	35
表格三十一、考量動態配置 AWL 下 Cifar-10 中 ADC Latency	35
表格三十二、考量動態配置 AWL 下 VGG-8 中 ADC 準確率	36
表格三十三、考量動態配置 AWL 下 VGG-8 中 ADC Latency	36

第一章、緒論

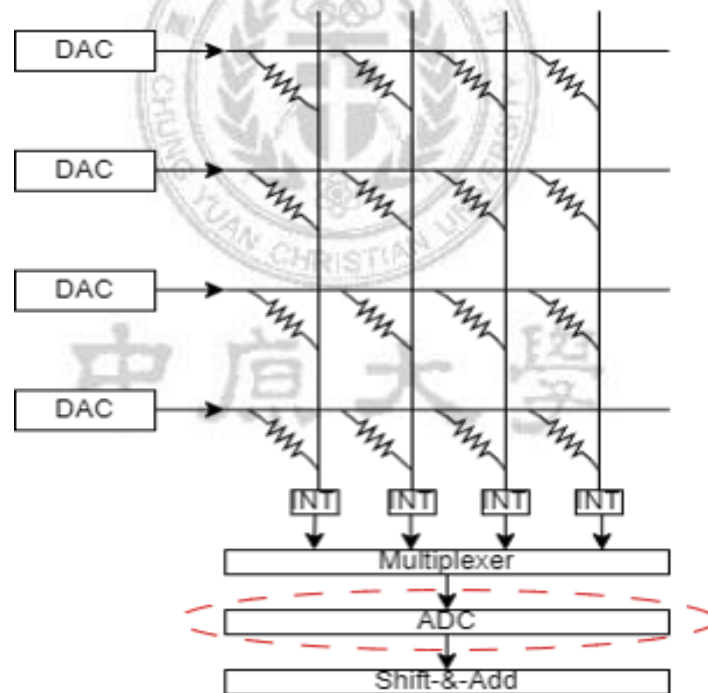
卷積神經網絡 (Convolutional Neural Network, CNN) 是深度學習領域中的關鍵技術，在機器學習、圖像識別以及其他多種應用[1][2]中展現出卓越的效能。然而，隨著網絡結構變得愈加深層和複雜，對計算資源和能源的需求也隨之增加，尤其是在計算和能源受限的設備上。為了應對這些挑戰，CIM (Computing in-Memory) 架構應運而生[3][4][5]。CIM 利用記憶體裝置的物理特性直接在記憶體中進行計算，不僅加快了計算速度，還降低了能源消耗。因此，CIM 被認為是一種非常具有潛力的解決方案，顛覆了傳統的計算模型，提供了一種全新的計算方式。



圖一、CIM 架構[6]

CIM 架構的核心優勢在於其能夠利用記憶體裝置的物理特性，在記憶體本身內部直接進行計算操作。這種方法大幅減少了數據在記憶體和處理器之間的移動需求，不僅加快了計算速度，還有效降低了能源消耗。在 CIM 架構中，神經網絡的權重被直接儲存在記憶體單元中，並通過類比的方式進

行權重和輸入數據的乘法運算[6]。如圖一所示，CIM 架構利用電阻來代表權重，輸入電壓則代表輸入信號。這種類比計算方式可以同時在多個記憶單元上進行，從而大幅度地減少了能源消耗。之後，這些類比信號需要被轉換回數位信號，以便進行進一步的處理。通過將計算過程直接嵌入到記憶體單元中，實現了資料和計算的緊密集成，避免了傳統計算模型中數據頻繁在記憶體和處理單元間傳輸所引起的瓶頸問題。這不僅提高了系統的計算效率，還顯著降低了能量的消耗。傳統的 CNN 運算通常依賴於大量的矩陣乘法和加法操作，這些操作需要頻繁訪問記憶體數據並在處理單元中進行計算。而 CIM 架構利用記憶體的固有特性，將這些運算直接在記憶體內部完成，使得 CNN 的運算過程更為高效。



圖二、在 CIM 架構中的 ADC[7]

圖二則進一步展示了數位轉換器 (Analog-to-digital converter, ADC) 在 CIM 架構中的關鍵作用[7]。ADC 是類比計算與數位處理之間的橋梁，其位元精度會直接影響到轉換過程的精確度。高位元精度能夠提供更精確的數

位輸出，但這同時也意味著更高的能源消耗和更長的轉換時間。相反，較低的位元精度可以降低能源消耗並加快轉換過程，但卻有可能因為量化誤差的發生而導致計算的精度下降。因此，找到 ADC 位元精度與能效和計算精度之間的平衡點便成為了重點。

除此之外，記憶體內的 Word-Line 激活也是影響 CIM 運算效率的另一重要因素。在理想狀態下，激活更多的 Word-Line 可以同時進行更多的計算，但這同時也會增加由於記憶體單元之間的交互作用導致的誤差。尤其是在使用 RRAM (Resistive Random Access Memory)[8]作為運算記憶體時，RRAM 有讀取快速、低面積成本、近零靜態功率消耗等優點[9]，因此適合作為 CIM 的運算記憶體，但是在過多的 Word-Line 激活狀態下可能會因為高阻態單元產生的微弱電流而導致計算誤差的出現[10]。因此，需要合理地限制同時激活的 Word-Line 數量，並且根據網絡的不同層需要進行分層調整，這也是可以提高 CIM 效率的另一個重點。

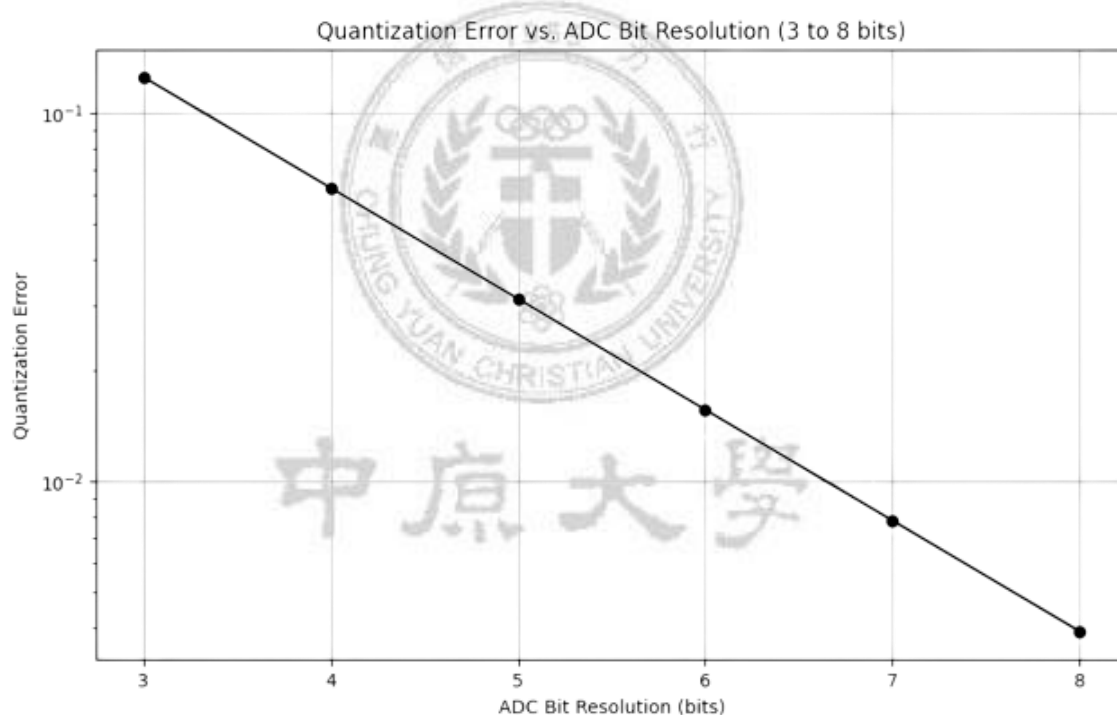
在目前的研究中，對於 CIM 架構中的 AWL 和 ADC 位元精度已經有不少的研究。這些研究分別探討了如何通過優化記憶體內部的 AWL 來提高計算效率和精度，以及如何調整 ADC 的位元數以達到能源效率和計算精度之間的最佳平衡。然而，目前透過整合這兩者來研究 CIM 架構的效率和效能的文獻仍然相稀少。

因此本篇論文將整合 AWL 及 ADC 的調整策略，我們提出了一種分層調整字線激活和 ADC 位元精度的方法，在保證準確率的同時如何達到能效最大化。

第二章、背景研究

2.1 ADC 位元精度對準確率的影響

在深度學習模型中，每增加一個位元，ADC 就能更細緻地量化類比信號，更準確地捕捉輸入數據的細節和特徵[11]。這對於提高諸如圖像識別、語音識別等任務的模型性能非常重要。因為更精確的數據可以幫助模型學習的更仔細，從而提高整體分類或預測的準確率。



圖三、ADC 位元精度與量化誤差關係

增加 ADC 的位元精度也意味著量化級別會跟著增加，這讓每增加一位位元，數位信號能夠表示的數據範圍會加倍。如圖三所示這種增加有助於減少量化誤差，而量化誤差就是一個影響準確率的重大關鍵，量化誤差是由將連續類比信號轉換成數位信號的過程所產生的。量化過程中，每一

個量化級別實際上代表了一個範圍的類比值，類比值在該範圍內的任何一點都被映射到該級別的數位值。透過 $\text{Quantization Error} = \frac{1}{2^{bits}}$ 的計算方式我們可以發現當位元數較低時，量化誤差就會越大，而量化誤差的形成便會阻礙模型準確地學習和預測，進而影響最終的準確率[12]。因此，提高 ADC 的位元精度是提升模型表現的一個方法。

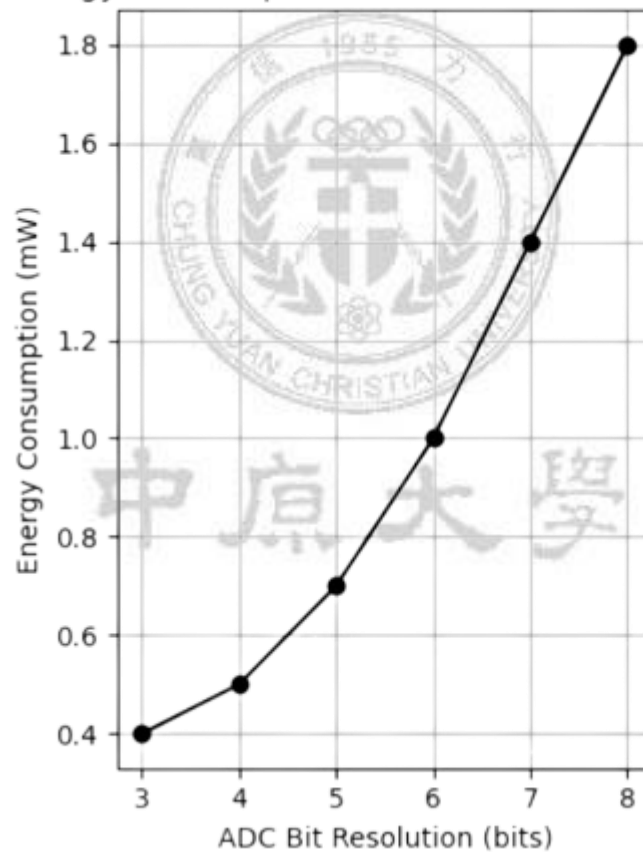
但是並非越高的位元精度越好。當超過某個閾值後，準確率提高的效果就會變低，但是計算的複雜度和能源消耗卻會顯著增加。在實際應用中，這可能會導致硬體成本的上升，並且還會增加處理器的負擔，特別是在資源有限的嵌入式系統中。因此，找到 ADC 位元精度的最佳平衡點，能夠在確保不過度消耗資源的情況下保持準確率才是較好的方法。

在設計和部署 CNN 模型時，仔細考慮每一層所需的位元精度，並且根據不同層的特定需求進行配置就會變得很重要。例如，在特徵提取層，可能需要較高的位元精度以捕捉更多細節；而在其他層，如分類層，較低的精度可能就已經足以應付。

2.2 ADC 位元精度對能耗與延遲的影響

在 CIM 架構中，ADC 的位元精度除了對準確率會有影響之外，對能源的消耗與延遲也同時有著很大的影響。在 CIM 中，ADC 負責將類比信號轉換成數位信號，ADC 的位元精度決定了信號解析度的高低。更高的位元精度意味著類比信號的細節能夠被更精確地捕捉，這雖然能夠提高模型的準確率，但與此同時也會導致更高的能源消耗與延遲[7]。

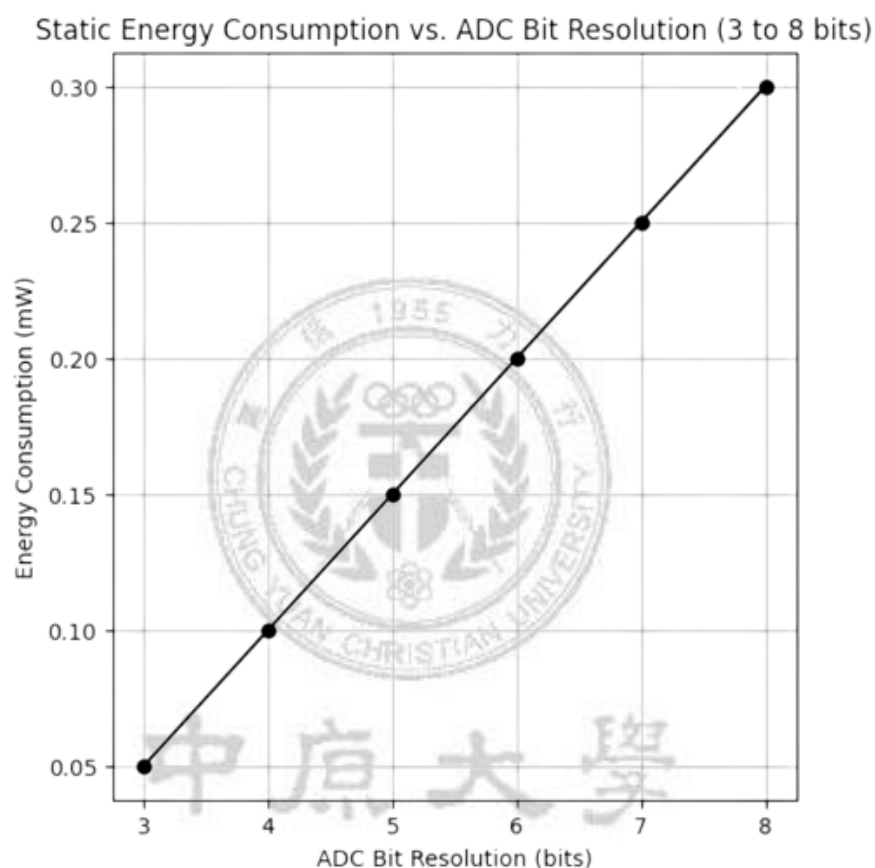
Dynamic Energy Consumption vs. ADC Bit Resolution (3 to 8 bits)



圖四、ADC 位元精度與動態能耗關係

ADC 的能量消耗可分為動態的能量消耗和靜態的能量消耗。如圖四顯示了 ADC 位元精度與動態能耗關係，我們可以看到 ADC 的位元精度越高

能量的消耗也會跟著變高。這是因為動態的能量消耗主要與 ADC 進行量化操作的次數相關，每次量化操作所需的能量會隨著位元精度的提高而增加。越高位元精度的 ADC 就會需要更複雜的電路來處理信號中的細微變化，這就增加了計算負荷，也增加了能量消耗。



圖五、ADC 位元精度與靜態能耗關係

另一方面如圖五，靜態的能量消耗也與 ADC 的位元精度相關，因為即使在沒有進行量化時，高位元精度的 ADC 比起低位元精度的 ADC 也會需要消耗更多能量來維持 ADC 的狀態。

除此之外，ADC 的位元精度對延遲的影響也不容忽視。位元精度越高，意味著每次轉換所需的時間也會越長，從而增加了延遲。這是因為更高精度

的 ADC 需要更多的時間來精確地量化每個輸入信號，這種延遲可能會對整體系統性能產生顯著影響。高位元精度的 ADC 由於需要更多的比較器和更複雜的數位邏輯，導致每次量化操作的時間增加。這些增加的延遲在需要實時處理的應用中特別重要。所以在選擇 ADC 位元精度時，不僅需要考慮能量消耗，還需要權衡延遲帶來的影響，以確保系統能夠在滿足準確率要求的同時，保持合理的響應速度。

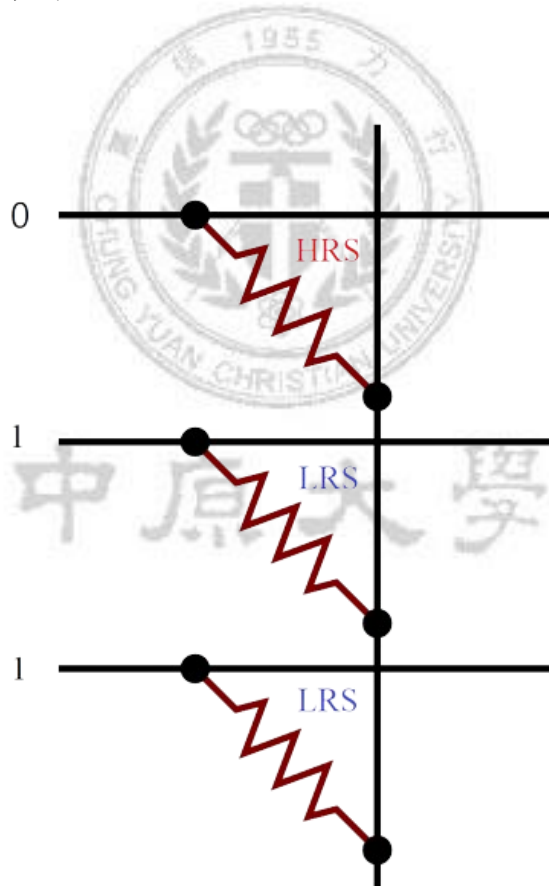
因此，選擇合適的 ADC 位元精度對於在 CNN 的能量效率、延遲與準確率之間取得平衡也是一個重要的課題。儘管高位元精度可以提高模型的準確性，但是卻可能會導致不必要的能源消耗以及延遲。相反，降 ADC 的低位元精度雖然可以減少能量消耗與延遲，但是由於量化誤差的增加，可能也會導致模型的準確率降低。所以，根據不同的應用需求靈活調整 ADC 的位元精度會是一種比起固定 ADC 位元精度還要有效的做法。

例如，在需要處理高分辨率圖像或複雜視頻數據的圖像識別應用中，對於細節要求高的部分（如面部識別或物體追蹤），使用高精度 ADC 雖然會比較耗能，但是為了避免太多的誤差產生這個時候使用高精度 ADC 會比較合適；而對於背景或不重要的部分就可以反過來使用較低精度的 ADC，這樣就可以在保障準確率的同時，也能降低總體能量消耗。

根據上述的研究可以發現 ADC 的位元精度對 CIM 架構中的能量消耗和性能有著重大的影響。因此通過動態配置 ADC 的位元精度，我們可以在性能和能效之間找到最佳平衡，滿足不同的需求。

2.3 電流偏差與 Word-Line 激活數量問題

在進行 CIM 運算時，如何處理資料和權重的轉換及其對運算準確率的影響也是挑戰之一。在 CIM 架構中，輸入特徵映射（Input Feature Map, IFM）會被轉換成 0 或 1 的電壓，並從 Word-Line 端施加到 CIM 陣列中。除此之外，權重也會被轉換為二進位形式，並映射進入相同的 CIM 陣列中。CIM 的運算記憶體單元利用不同的阻抗狀態來代表二進位資料：低阻抗狀態（Low Resistance State, LRS）表示 1，高阻抗狀態（High Resistance State, HRS）表示 0。

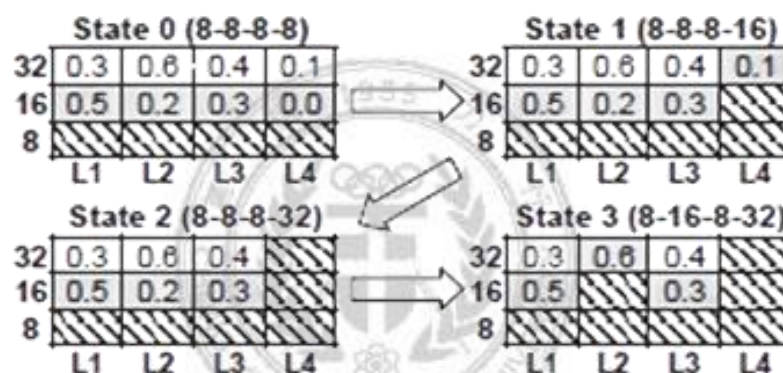


$$I_{ideal} = 2 \times I_{LRS}$$

$$I_{total} = I_{HRS} + 2 \times I_{LRS}$$

圖六、CIM 架構下的電流偏差[13]

理想情況下，代表 0 的 HRS 應該不會產生任何電流（即 $I_{HRS} = 0$ ）。然而，在實際應用中，即便是代表 0 的 HRS，依然會產生微弱的電流。這種現象如果在大規模的 CIM 陣列中被累積，則有可能導致計算誤差。如圖六，在一個 Bit Line 上，一個單元處於 HRS 而其餘兩個單元處於 LRS，則理想情況下總電流會是 $I_{ideal} = 2 \times I_{LRS}$ 。然而在實際上總電流會是 $I_{total} = I_{HRS} + 2 \times I_{LRS}$ ，其中多出的 I_{HRS} 就有可能對計算結果造成不利的影響。為了解決這一問題，CIM 運算設計中的一個策略是控制 AWL 的數量。



圖七、AWL 決策方法[6]

如圖七，有些研究透過貪婪演算法的策略來最小化這種誤差[6]。這種方法首先分層調整 AWL 的數量，通過連續的實驗來確定每層應該要激活的 Word-Line 數量。在這些實驗中，他們會監測不同配置對計算準確率的影響，並選擇造成最小準確率下降的配置進行下一輪調整。例如，如果激活 16 行 Word-Line 的配置導致了最小的準確率下降，則這一配置會被用於後續實驗，直到達到一個預設的誤差閾值為止。

2.4 整合 ADC 位元精度和 Word-Line 激活數量策略

在上述研究中，我們已經深入探討了在 CIM 架構中單獨研究 ADC 位元精度和 AWL 數量的影響，分析了這些因素對運算準確性、能耗以及計算效率的具體影響。這些研究說明了 ADC 位元精度和 AWL 數量會如何獨立影響 CIM 的性能。然而，這些研究主要集中於單一方面的優化，並未探討將這兩個因素整合應用時的效果。這兩者的相互作用對系統的運算效能和能效會產生顯著影響。因此，研究結合這兩個因素將有助於發展更全面的策略，以實現高效能與高能效的雙重目標。

進一步的整合研究將有助於在不同條件下，如何通過調整 ADC 位元精度和 AWL 數量來達到最佳性能。因此，在接下來來的研究中，我們將探索這些技術因素的整合使用，透過理論分析與實驗驗證，來確定各種配置下的性能和能效關係。

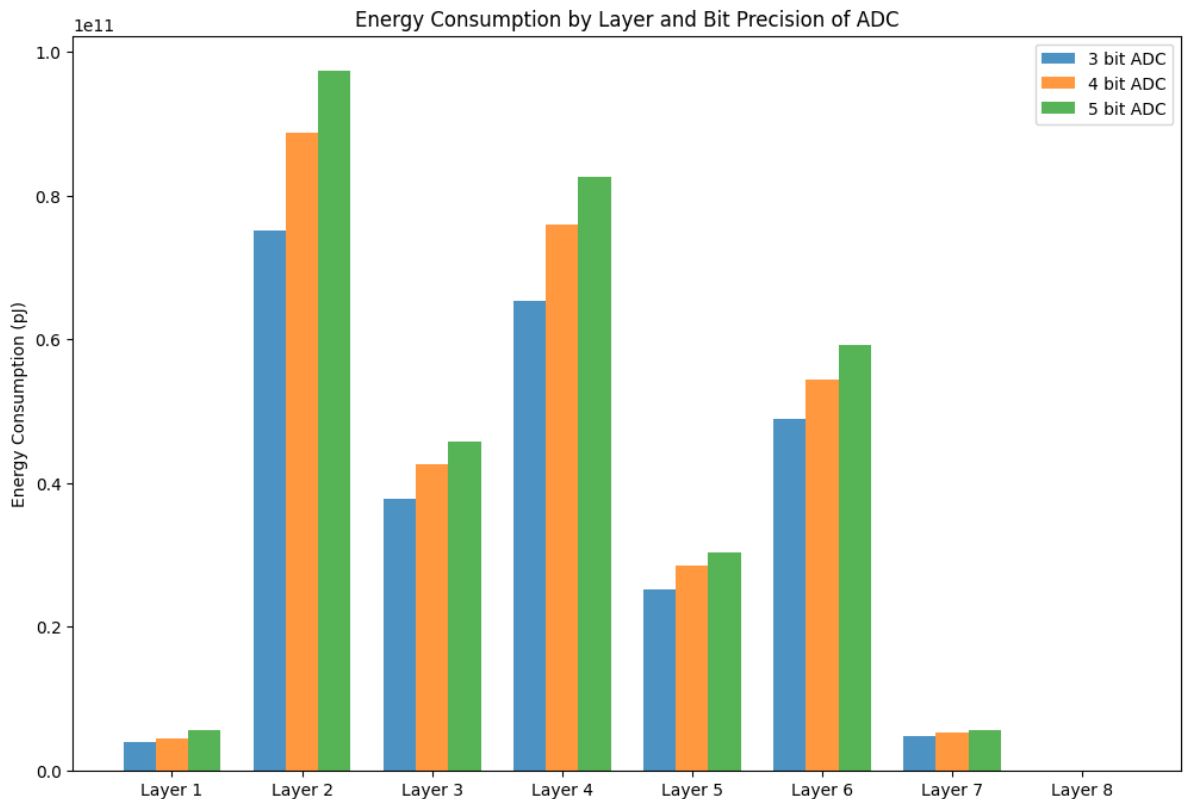
中原大學

第三章、研究方法

3.1 研究動機與問題闡述

3.1.1 ADC 位元精度

在使用 CIM 架構下的 CNN 中，其模擬與數據運算過程直接在記憶體中進行，這能顯著降低數據傳輸的需求，從而降低整體能耗以及延遲。而 ADC 在這個架構中扮演了重要角色，ADC 負責將類比信號轉換為數位信號，以供數位處理器處理。而所謂的位元精度是指 ADC 在轉換過程中能表達的最小信號細節的數據位數，它直接影響了數據的精確性、系統的能耗與延遲。



圖八、不同 ADC 位元精度在每一層的能耗比較

透過使用 VGG-8 進行模擬後，從圖八中我們可以看到高位元精度的 ADC 在每一層的能量消耗中相較於低位元精度的 ADC 高都還要高出不少，透過表格一可以得知在最終的總能耗中 5 bit ADC 比 3 bit ADC 高出了 25.06%。

表格一、不同位元精度 ADC 在各層的能耗表現

	3 bit ADC	4 bit ADC	5 bit ADC
Layer 1 Energy	3.912+09pJ	4.507e+09pJ	5.543e+09pJ
Layer 2 Energy	7.519+10pJ	8.874e+10pJ	9.737e+10pJ
Layer 3 Energy	3.774+10pJ	4.253e+10pJ	4.574e+10pJ
Layer 4 Energy	6.530+10pJ	7.605e+10pJ	8.262e+10pJ
Layer 5 Energy	2.523+10pJ	2.853e+10pJ	3.026e+10pJ
Layer 6 Energy	4.891+10pJ	5.433e+10pJ	5.921e+10pJ
Layer 7 Energy	4.727+09pJ	5.183e+09pJ	5.625e+09pJ
Layer 8 Energy	1.231+07pJ	1.351e+07pJ	1.411e+07pJ
Total Energy	2.610+11pJ	2.999+11pJ	3.264+11pJ

表格二、不同位元精度 ADC 的延遲

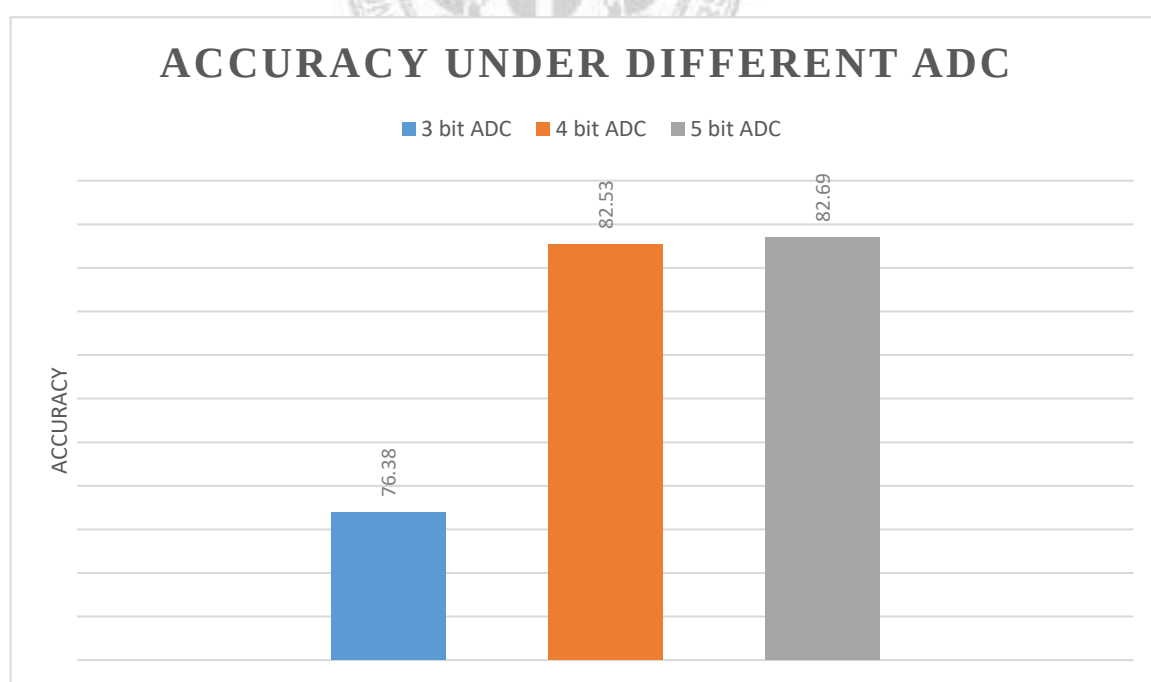
	3 bit ADC	4 bit ADC	5 bit ADC
ADC Latency	5.49	6.60	7.21

延遲的部分透過使用 VGG-8 進行模擬後，我們可以從表格二發現高位

元精度的 ADC 由於需要更多的計算資源和時間來完成量化，因此比起低位元精度的 ADC 會有更高的延遲。

除了在能耗與延遲方面，位元精度的選擇對模型的準確率也有顯著影響。高位元精度的 ADC 比起低位元精度的 ADC 更能夠更精確地表達類比信號的細節，這對於 CNN 模型在學習和預測過程中是非常重要的。因為更高的精度也就意味著在信號轉換過程中的信息損失更少，這有助於模型捕捉到更細微的特徵。

透過圖九及表格三，我們可以觀察到，使用 5 bit ADC 的情況下相較於 3 bit ADC 的模型準確率高了 6.31%，明確表示了高位元精度的 ADC 更能夠準確地識別目標。這是因為更高位元的 ADC 提供了更多級別的信號量化，允許模型在學習的過程中對輸入數據有更深的解析和反應。



圖九、不同位元精度的準確率比較圖

表格三、不同位元精度 ADC 的準確率

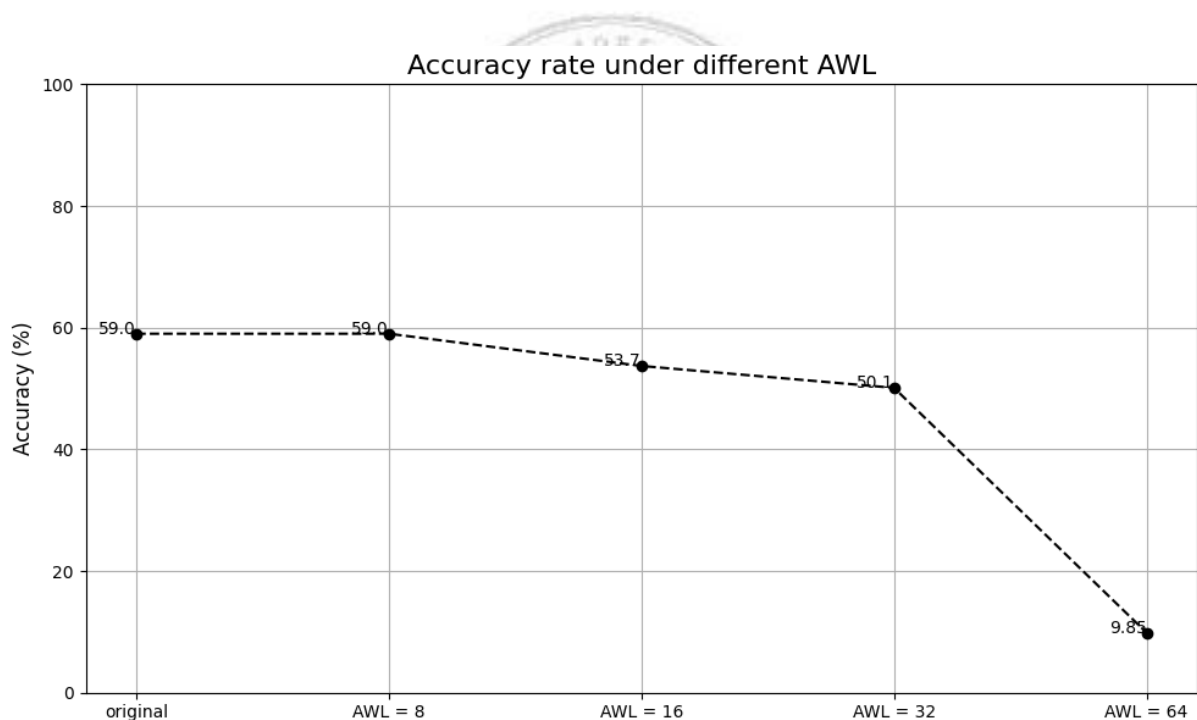
	3 bit ADC	4 bit ADC	5 bit ADC
Accuracy	76.38	82.53	82.69

所以我們就面臨了一個能耗與準確率的取捨問題。雖然高位元精度可以提升模型的性能，但同時也會增加硬體的成本和能耗。因此，在設計 CIM 架構的 CNN 系統時，就必須考慮到實際應用的需求和成本效益比，選擇最適合的 ADC 位元數。



3.1.2 字線激活數量

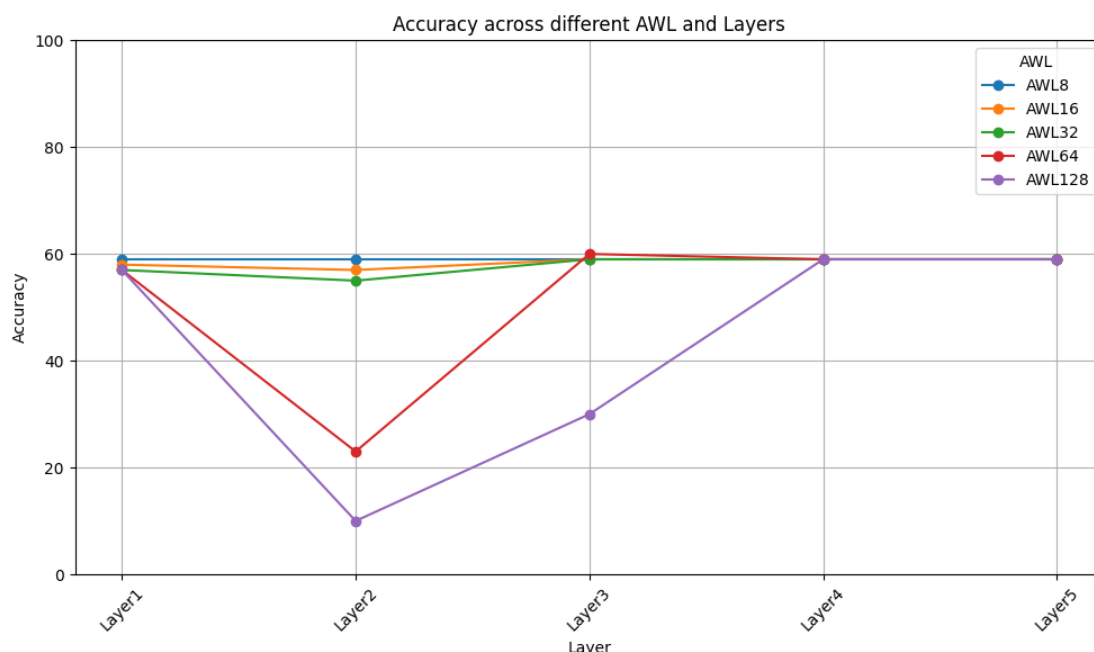
當運算記憶體在運行時，因電流偏差而引發的 I_{HRS} 問題會隨著同時激活的 Word-Line 數量增加而加劇，從而導致更大的誤差。雖然通過限制 AWL 數量不超過電阻開關比可以有效減少這種誤差，但是這種方法在提高準確率的同時卻會降低了計算效率。因此，儘管限制 AWL 能控制誤差，但這種策略在提升運算速度方面則效果不佳。但是一昧的提高 AWL 的數量，雖然可以有效提升計算速度，但這樣做往往會犧牲準確率。



圖十、AWL 對準確率的影響

如圖十，在使用我們使用 Cifar-10 模型時，可以發現在固定 ADC 位元精度只調整 AWL 數量的方法下，隨著 AWL 的增加，雖然能夠將計算時間大幅縮短，但是我們同時卻也發現了準確率開始下降。特別是當 AWL 為 64 時，計算時間雖然縮短至原來的八分之一，但準確率卻已經由原本

的 59% 降至 9.85%。這顯示如果在整個網路都使用相同的 AWL 時想要藉此追求效率的同時，會受限於其可靠性。

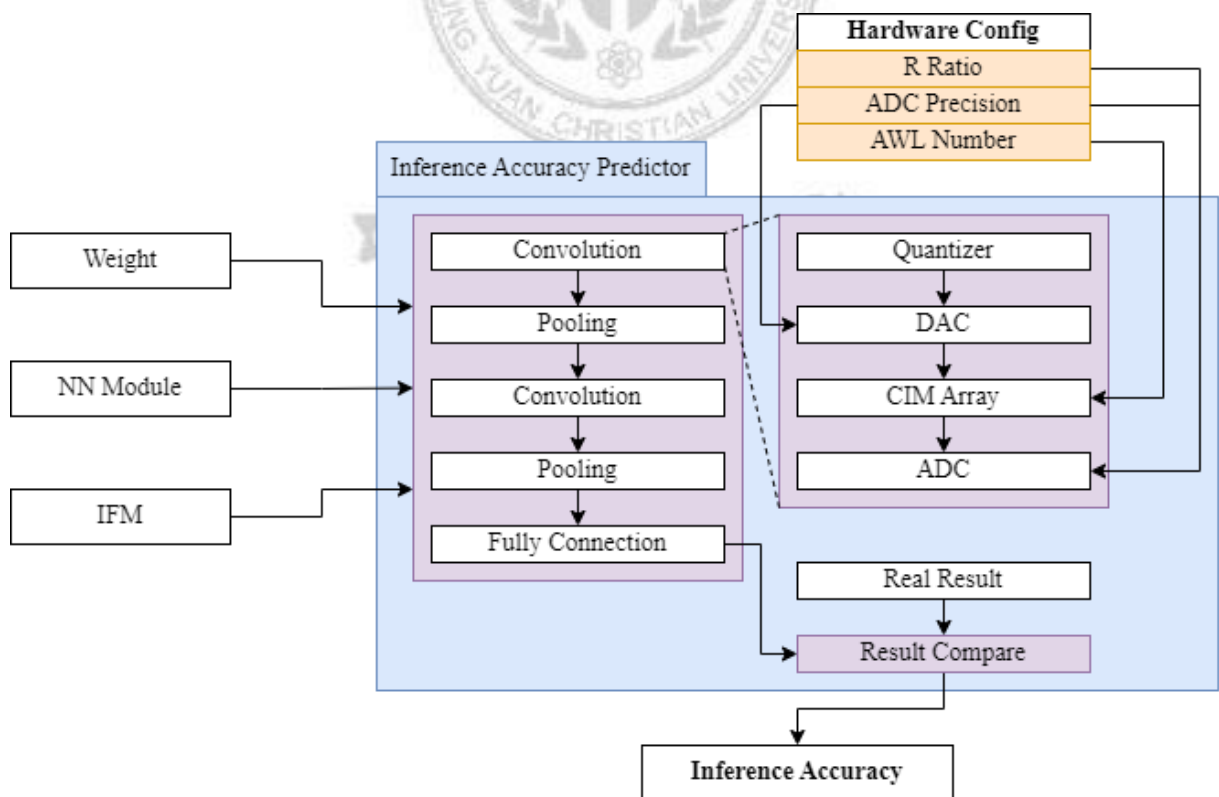


圖十一、AWL 對每一層的個別影響

當我們透過逐層調整 AWL 並觀察結果後，我們可以發現其實 AWL 對於每一層的影響都不盡相同。如圖十一，我們發現第二層由於其權重矩陣的行數最多，因此對 AWL 的改變反應最為強烈。相比之下，第三層的權重矩陣行數較少，對 AWL 的變化敏感度也較低；而第一、四、五層的權重矩陣行數更少，對 AWL 的變化幾乎只有輕微的影響。透過這些數據顯示，AWL 的變化對權重矩陣行數多的層影響較大。因此，我們可以選擇在對 AWL 變化較不敏感的層使用較高的激活數量，而在對 AWL 變化敏感的層則保持較低的激活數量以維持準確性。

3.2 CIM 模擬工具

為了能進行我們的實驗，我們會需要一個能夠讓我們進行逐層調整 ADC 位元精度與 AWL 數量的模擬工具。但由於目前許多 DNN 模擬工具，如 DL-RSIM[16]、NeuroSIM[17][18]、RAPIDNN[19]、PUMA[20] 等。上述的那些工具都有各自專精的領域，不幸的是正好沒有能符合我們要求的功能。因此為了能夠精確模擬符合我們要求的 CIM 架構，我們開發了一套稱為 Inference Accuracy Predictor 的工具[13]，此系統在 Windows 10、Anaconda 23.3.1、以及 Python 3.10.9 的開發環境中構建而成。如圖十二，此工具可以依據不同的硬體條件，如 R ratio、AWL、及 ADC 位元數，來評估 CIM 計算中可能產生的誤差，並估算在給定條件下的準確率。



圖十二、模擬工具架構

如圖十二便是我們模擬工具的架構圖，在使用 Inference Accuracy Predictor 之前，需要進行一系列的準備工作。首先，利用 Keras 訓練神經網絡，然後保存訓練後的模型權重。根據提供的架構圖，這些訓練後的權重和網絡架構，連同測試數據集和具體的硬體配置（包括 R ratio、AWL、ADC 精度），會作為輸入提供給 Inference Accuracy Predictor。

在模型運行過程中，數據首先通過卷積層，這些層負責提取圖像的特徵；接著是池化層，用於降低參數數量並保留重要信息，這有助於減少運算負擔和避免過度擬合。之後，數據經過進一步的卷積和池化處理，最終到達全連接層，這一層將前面提取的特徵綜合起來，生成最終的結果。



3.3 逐層考量字線激活數量與 ADC 位元精度決策方法

在進行研究時，我們發現多數的研究都是對 AWL 跟 ADC 的個別研究因此本論文中我們將同時考慮 AWL 與 ADC 去做逐層的配置。首先，我們探討了許多影響神經網絡 AWL 設定的關鍵因素。R ratio 的大小是一個重要的考慮點，當 R ratio 較大時，系統就越能夠承受電流偏差。再來，神經網絡中的不同層對於 AWL 的需求並不相同。例如，位於網絡前端的卷積層因為需要較高的計算準確度，以確保後面層次的輸出正確就會偏向使用較小的 AWL。而位於後端的全連接層則對準確率的要求相對較低，因此就比較能夠使用較大的 AWL。除此之外，每一個輸出像素點所需進行的點積運算由權重行數反映，行數增多時，累積的電流偏差也可能增加。最後，權重中的 0 的比例也會影響計算，0 的比例高意味著更多計算單元處於高組態，從而可能引起較大的電流誤差。根據上述的研究我們透過表格四將這些因素轉化為數值，並稱為風險因子，並根據各層特性計算相應的風險指數[13]。

表格四、AWL 的風險指數評估策略[13]

Risk Factor	Value
R ratio ≤ 10	+1
Convolution Layer	+1
Matrix Row (R) ≥ 256	$+[\log_2 R - 7]$
Ratio of 0 in MSB $\geq 55\%$	+1
R ratio ≥ 20	-1

最後我們會參照表格五依據計算出的風險指數為每一層分配合適的 AWL 大小，以控制潛在誤差。對於風險較高的層，我們便會使用較小的 AWL 以減少誤差，而風險較低的層則可以採用較大的 AWL 以此來提高計算效率。

表格五、AWL 的配置對照表[13]

Risk Value	AWL
>3 (ratio ≤ 10)	8
≥ 3	16
2	32
1	64
< 0	128

至於 ADC 位元精度的決策上也採用類似的方法，如表格六及表格七所示。首先 ADC 在不同層的影響各有不同，卷積層主要負責從輸入數據中提取空間特徵，例如邊緣、紋理和形狀。這些層會直接處理原始輸入數據或其衍生的特徵映射，因此需要較高的位元精度以保持這些細微特徵的完整性。由於卷積層會直接影響後續層的輸入品質，使用較高位元精度的 ADC 將會有助於減少量化誤差，從而保護關鍵的信息不被丟失。全連接層則主要負責將已提取的特徵進行整合和分類。由於全連接層處理的是較為抽象的數據，這些數據已經透過前面的卷積層處理過，因此相比於卷積

層，全連接層可以容忍較低的 ADC 位元精度。再來在網絡的前幾層，因為他們通常都是在處理與原始輸入數據直接相關的低級特徵。因此他們會比較需要較高的精度以保持細節的完整性。而到了網絡的較深層，已經是在處理由前面層的輸出逐步合成較為抽象的特徵。由於這些特徵已經是由多個較低級的特徵組合而成，因此在 ADC 位元精度的選擇上就可以稍微降低。至於 Matrix Row 的部分在高於 512 時對於模型才会有較明顯的影響，因此我們將 Matrix Row 的風險計算調整為 $\lfloor \log_2 R - 8 \rfloor$ 。

表格六、ADC 的風險評估策略

Risk Factor	Value
First 1/3 Layer	+2
Convolution Layer	+1
Dense Layer	-1
Matrix Row (R) ≥ 512	$+\lfloor \log_2 R - 8 \rfloor$

表格七、ADC 的配置對照表

Risk Value	ADC bit
≥ 3	5
2	4
< 2	3

上述所提出的 ADC 與 AWL 風險指數計算都是單獨考量 ADC 或 AWL 的情況下的評估方法，因此接下來我們將提出另一種方法是考量 ADC 與 AWL 會互相影響的情況下該如何計算我們的風險指數與配置策略。

考量到同時進行動態配置 AWL 跟 ADC 會對結果造成較大的影響，我們提出了另一種 ADC 的決策方法，該方法會基於表格四與表格五的 AWL 決策方法去做 ADC 位元精度的選擇。如表格八所示。我們在原本的風險評估策略中加入了 AWL 的風險指數，因為 AWL 的配置會直接影響整體系統的容錯能力，從而間接影響到 ADC 所需的精度。當 AWL 風險指數較高時，意味著系統的容錯能力較低，因此需要更高精度的 ADC 來減少可能的量化誤差。相對地，如果 AWL 風險指數較低，則可以適當降低 ADC 的精度以節省資源。這樣的綜合風險評估策略能更好地平衡系統的準確性與效率。

表格八、考量 AWL 狀態下 ADC 的風險評估策略

Risk Factor	Value
First 1/3 Layer	+2
Convolution Layer	+1
Dense Layer	-1
Matrix Row (R) ≥ 512	$+[\log_2 R - 8]$
AWL Risk Value ≥ 4	+1

第四章、實驗結果

4.1 實驗設置

在本次實驗中，我們將利用先前開發的 Inference Accuracy Predictor，對 Cifar-10[14]和 VGG-8[15]進行實驗，目的是驗證我們提出的最佳化策略的實際效果。表格九為實驗的配置。

我們的研究重點在於分析 R ratio 為 10 及 15 的條件下，Word-Line 的分層激活數量對計算效能的影響，同時探討根據不同神經網絡層的特性，動態配置模擬的 ADC 位元精度如何影響整體的計算效率和準確率。

表格九、實驗配置

Simulator	Accuracy Predictor
CNN	Cifar-10、VGG-8
CIM array size	512 * 512
R ratio	10、15
ADC bit	3、4、5

首先是 Cifar-10 的架構，表格十、Cifar-10 架構為架構的詳細配置，其中 Matrix Row 代表展開後的權重行數，計算方式為 Kernel 的平方乘以 Channel 數，而 Matrix Column 則對應於 Kernel Depth。

表格十、Cifar-10 架構

	Input	Channel	Kernel	Depth	Matrix Row	Matrix Column
Layer 1	32	3	3	32	27	32
Layer 2	15	32	5	64	800	64
Layer 3	3	64	3	64	576	64
Layer 4	1	64	1	64	64	64
Layer 5	1	64	1	10	64	10

表格十一、Cifar-10 中 ADC 的風險因子

	Layer Type	Matrix Row
Layer 1	Conv	27
Layer 2	Conv	800
Layer 3	Conv	576
Layer 4	Dense	64
Layer 5	Dense	64

再來我們便會透過第三章提出的風險因子計算每一層配置 ADC 的風險指數。透過表格十一列出的風險因子我們可以得到表格十二，也就是我們 ADC 的最終配置，我們以 Layer 2 來當作例子透過表及表來計算風險指數 (1) 類型為卷積層 ($\text{Risk} = 1$) (2) Matrix Row ($R \geq 512$) ($\text{Risk} = \lfloor \log_2 800 - 8 \rfloor = 1$)，因此風險指數為 $(1)+(2) = 1 + 1 = 2$ 。得到風險指數後再比對表格七就可以得知我們在該層的 ADC 位元精度為 4。

表格十二、Cifar-10 中 ADC 的風險指數與配置

	Risk Value	ADC bit
Layer 1	3	5
Layer 2	2	4
Layer 3	2	4
Layer 4	-1	3
Layer 5	-1	3

表格十三、Cifar-10 中 AWL 的風險因子

	Layer Type	Matrix Row	Ratio of 0 in MSB
Layer 1	Conv	27	51.39 %
Layer 2	Conv	800	57 %
Layer 3	Conv	576	56.18 %
Layer 4	Dense	64	52.59 %
Layer 5	Dense	64	53.28 %

接著我們便會透過表格十三計算每一層配置 AWL 的風險。表格十四就是我們 AWL 的最終配置，我們以 Cifar-10 中 R ratio 為 10 的 Layer 2 來當作例子透過表及表來計算風險指數(1) R ratio ≤ 10 (Risk = 1) (2) 類型為卷積層(Risk = 1) (3) Matrix Row (R) ≥ 256 (Risk = $\lfloor \log_2 256 - 7 \rfloor = 2$) (4) Ratio of 0 in MSB $\geq 55\%$ (Risk = 1)，因此風險指數為(1)+(2)+(3)+(4) = 1 + 1 + 2 + 1 = 5。得到風險指數後再比對表格五就可以得知我們在該層的 AWL 為 8。

表格十四、Cifar-10 中 AWL 的風險指數與配置

	Risk Value		Active WL Number	
R Ratio	10	15	10	15
Layer 1	2	1	32	32
Layer 2	5	4	8	16
Layer 3	5	3	8	16
Layer 4	1	-1	64	64
Layer 5	1	-1	64	64

表格十五到表格十九則是 VGG-8 的架構、風險指數以及每一層 ADC 位元精度以及 AWL 數量。

表格十五、VGG-8 架構

	Input	Channel	Kernel	Depth	Matrix Row	Matrix Column
Layer 1	32	3	3	32	27	32
Layer 2	32	32	3	32	288	32
Layer 3	16	32	3	64	288	64
Layer 4	16	64	3	64	576	64
Layer 5	8	64	3	128	576	128
Layer 6	8	128	3	128	1152	128
Layer 7	1	2048	1	128	2048	128
Layer 8	1	128	1	10	128	10

表格十六、VGG-8 中 ADC 的風險因子

	Layer Type	Matrix Row
Layer 1	Conv	27
Layer 2	Conv	288
Layer 3	Conv	288
Layer 4	Conv	576
Layer 5	Conv	576
Layer 6	Conv	1152
Layer 7	Dense	2048
Layer 8	Dense	128

表格十七、VGG-8 中 ADC 的風險指數與配置

	Risk Index	ADC bit
Layer 1	3	5
Layer 2	3	5
Layer 3	1	3
Layer 4	2	4
Layer 5	2	4
Layer 6	3	5
Layer 7	2	4
Layer 8	-1	3

表格十八、VGG-8 中 AWL 的風險因子

	Layer Type	Matrix Row	Ratio of 0 in MSB
Layer 1	Conv	27	52.78 %
Layer 2	Conv	288	54.83 %
Layer 3	Conv	288	51.54 %
Layer 4	Conv	576	56.31 %
Layer 5	Conv	576	53.25 %
Layer 6	Conv	1152	56.76 %
Layer 7	Dense	2048	56.02 %
Layer 8	Dense	128	48.67 %

表格十九、VGG-8 中 AWL 的風險指數與配置

	Risk Index		Active WL Number	
R Ratio	10	15	10	15
Layer 1	2	1	32	32
Layer 2	3	2	16	32
Layer 3	3	2	16	32
Layer 4	5	4	8	16
Layer 5	4	3	8	16
Layer 6	6	4	8	16
Layer 7	6	4	8	16
Layer 8	1	-1	64	128

以上的 ADC 配置使用的是未考量到 AWL 的影響，因此接下來的實驗配置我們會使用基於動態配置 AWL 下的 ADC 位元精度決策方法，表格八去做實驗配置。

表格二十、Cifar-10 中考量 AWL 下 ADC 的風險因子

	Layer Type	Matrix Row	Risk Value R ratio 10	Risk Value R ratio 15
Layer 1	Conv	27	2	1
Layer 2	Conv	800	5	4
Layer 3	Conv	576	5	3
Layer 4	Dense	64	1	-1
Layer 5	Dense	64	1	-1

表格二十一、Cifar-10 中考量 AWL 下 ADC 的風險指數與配置

	Risk Value R ratio 10	ADC bit	Risk Value R ratio 15	ADC bit
Layer 1	3	5	3	5
Layer 2	3	5	3	5
Layer 3	3	5	2	4
Layer 4	-1	3	-1	3
Layer 5	-1	3	-1	3

表格二十二、VGG-8 中考量 AWL 下 ADC 的風險因子

	Layer Type	Matrix Row	Risk Value R ratio 10	Risk Value R ratio 15
Layer 1	Conv	27	2	1
Layer 2	Conv	288	3	2
Layer 3	Conv	288	3	2
Layer 4	Conv	576	5	4
Layer 5	Conv	576	4	3
Layer 6	Conv	1152	6	4
Layer 7	Dense	2048	6	4
Layer 8	Dense	128	1	-1

表格二十三、VGG-8 中考量 AWL 下 ADC 的風險指數與配置

	Risk Value R ratio 10	ADC bit	Risk Value R ratio 15	ADC bit
Layer 1	3	5	3	5
Layer 2	3	5	3	5
Layer 3	1	3	1	3
Layer 4	3	5	3	5
Layer 5	3	5	2	4
Layer 6	4	5	4	5
Layer 7	3	5	3	5
Layer 8	-1	3	-1	3

4.2 實驗結果

首先看到表格二十四、表格二十五及表格二十六，這是我們在 Cifar-10 不考慮彼此影響下，使用逐層配置 AWL 與 ADC 位元精度下準確率、ADC 的能效、Cycle 數的比較。我們可以發現當我們在 R Ratio 為 10 時使用我們的策略比起全部使用最高位元精度的狀況損失了 6.79%的準確率比起次高的損失了 0.29%，但是我們在 ADC 上面的 Latency 卻分別少了 14.81%以及 6.9%。在 R Ratio 為 15 時，在使用我們的逐層配置 ADC 位元精度策略時準確率甚至在比全部使用次高位元精度的 ADC 還要高出 0.14%的同時減少了 5%的 ADC Latency。而在 Cycle 數方面比起全部 AWL 為 8 不會產生誤差的情況下分別減少了 9%和 54%。

表格二十四、Cifar-10 中固定 ADC 與逐層配置 ADC 準確率

AWL	Dynamic							
R Ratio	10				15			
ADC	3	4	5	Dynamic	3	4	5	Dynamic
ACC	9.83	21.25	27.75	20.96	31.75	42.37	49.94	42.51

表格二十五、Cifar-10 中固定 ADC 與逐層配置 ADC Latency

AWL	Dynamic							
R Ratio	10				15			
ADC	3	4	5	Dynamic	3	4	5	Dynamic
ADC Latency	6.79	7.97	8.71	7.42	7.96	8.80	9.44	8.36

表格二十六、Cifar-10 固定 AWL 與逐層配置 AWL 的 Cycle 數比較

R Ratio	10	10	15
AWL	8	Dynamic	Dynamic
ADC	Dynamic		
Layer 1 Cycle	4	1	1
Layer 2 Cycle	100	100	50
Layer 3 Cycle	72	72	36
Layer 4 Cycle	8	1	1
Layer 5 Cycle	8	1	1
Total Cycle	192	175	89
Benefit	X	-9 %	-54 %

再來看到表格二十七、表格二十八及表格二十九，則是 VGG-8 的比較。我們可以發現在 VGG-8 的表現中我們的策略有著更好的發揮。在 R Ratio 10 跟 15 時我們的配置跟全部使用最高位元精度的配置相比損失的準確率僅有 1.44%及 0.52%。但是在 ADC Latency 上卻減少了 14.98%及 11.06%。而在 Cycle 數上則是分別減少了 5%跟 52%。

表格二十七、VGG-8 中固定 ADC 與逐層配置 ADC 準確率

AWL	Dynamic							
R Ratio	10				15			
ADC	3	4	5	Dynamic	3	4	5	Dynamic
ACC	54.32	57.76	58.68	57.24	55.65	59.51	60.01	59.49

表格二十八、VGG-8 固定 ADC 與逐層配置 ADC Latency

AWL	Dynamic							
R Ratio	10				15			
ADC	3	4	5	Dynamic	3	4	5	Dynamic
ADC Latency	5.49	6.60	7.21	6.13	6.59	7.32	7.96	7.08

表格二十九、VGG-8 固定 AWL 與逐層配置 AWL 的 Cycle 數比較

R Ratio	10	10	15
AWL	8	Dynamic	Dynamic
ADC	Dynamic		
Layer 1 Cycle	4	1	1
Layer 2 Cycle	36	18	9
Layer 3 Cycle	36	18	9
Layer 4 Cycle	72	72	36
Layer 5 Cycle	144	144	72
Layer 6 Cycle	288	288	144
Layer 7 Cycle	512	512	256
Layer 8 Cycle	16	2	1
Total Cycle	1108	1055	528
Benefit	X	-5 %	-52 %

透過上述在 Cifar-10 以及 VGG-8 的實驗中我們可以發現，透過我們逐層配置 AWL 數量以及 ADC 位元精度的策略雖然對我們的準確率有著些微的影響，但是在犧牲一些準確率的情況下我們卻可以有效的降低 ADC 的 Latency 以及計算時的 Cycle 數。特別是在 VGG-8 的表現最為亮眼，成功的做到了在損失 1%左右的準確率下，減少了 ADC Latency 以及 Cycle 數的效果。

最後我們再來看到在考量動態配置 AWL 的影響下的 ADC 動態配置方法。表格三十到表格三十三是在 Cifar-10 與 VGG-8 中有考量 AWL 的影響以及不考慮 AWL 影響的準確率與 ADC 延遲比較，我們可以看到在考慮 AWL 影響下去調整 ADC 的位元精度可以在犧牲一些延遲的情況下有效的減少準確率的損失。

表格三十、考量動態配置 AWL 下 Cifar-10 中 ADC 準確率

AWL	Dynamic			
R Ratio	10		15	
ADC	不考慮 AWL	考慮 AWL	不考慮 AWL	考慮 AWL
ACC	20.96	23.07	42.51	44.68

表格三十一、考量動態配置 AWL 下 Cifar-10 中 ADC Latency

AWL	Dynamic			
R Ratio	10		15	
ADC	不考慮 AWL	考慮 AWL	不考慮 AWL	考慮 AWL
ADC Latency	7.42	8.01	8.36	8.79

表格三十二、考量動態配置 AWL 下 VGG-8 中 ADC 準確率

AWL	Dynamic			
R Ratio	10		15	
ADC	不考慮 AWL	考慮 AWL	不考慮 AWL	考慮 AWL
ACC	57.24	57.88	59.49	59.63

表格三十三、考量動態配置 AWL 下 VGG-8 中 ADC Latency

AWL	Dynamic			
R Ratio	10		15	
ADC	不考慮 AWL	考慮 AWL	不考慮 AWL	考慮 AWL
ADC Latency	7.42	8.01	8.36	8.79

中原大學

第五章、結論

本論文提出了 CIM 的分層調整 AWL 和 ADC 位元精度的策略，並透過先前開發的 CIM 模擬平台來進行模擬。我們透過找出會影響準確率的風險因子後，透過計算出的風險指數來評估配置在每一層 AWL 的數量以及 ADC 的位元精度。

而我們的實驗結果顯示了，我們的方法與使用固定 AWL 數量與 ADC 位元精度相比，我們在 Cifar-10 中雖然犧牲了 7.43% 的準確率，但同時卻減少了 11.44% 的 ADC Latency 以及 54% 的 Cycle 數。而在 VGG-8 中表現得更為亮眼，我們在 R Ratio 為 15 時以減少僅僅 0.52% 的準確率，減少了 11.06% 的 ADC Latency 與 52% 的 Cycle 數。另外我們還發現若是我們在做決策時有去考慮另一方的影響，可以更有效的在提升能效的同時去減少在準確率上的損失。透過上述的這些結果說明了我們的方法確實是一個可行且具有前瞻性的方案。

中原大學

參考文獻

- [1] A. Krizhevsky, I. Sutskever and G. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," Proc. Neural Information and Processing Systems, 2012.
- [2] L. Deng et al., "Recent advances in deep learning for speech research at Microsoft," 2013 IEEE International Conference on Acoustics, Speech and Signal Processing, 2013, pp. 8604-8608, doi: 10.1109/ICASSP.2013.6639345.
- [3] N. Verma, H. Jia, H. Valavi, Y. Tang, M. Ozatay, L-Y. Chen, B. Zhang and P. Deaville, "In-Memory Computing: Advances and prospects," IEEE Solid-State Circuits Magazines, pp. 43-55, 2019.
- [4] A. Shafiee, A. Nag, N. Muralimanohar, R. Balasubramonian, J. P. Strachan, M. Hu, R. S. Williams and V. Srikumar, "ISAAC: A Convolutional Neural Network Accelerator with In-Situ Analog Arithmetic in Crossbars," in ISCA, 2016.
- [5] S. Yu, "Neuro-inspired computing with emerging nonvolatile memorys," in Proceedings of the IEEE, vol. 106, no. 2, pp. 260-285, Feb. 2018, doi: 10.1109/JPROC.2018.2790840.
- [6] Y. Park, S. Y. Lee, H. Shin, J. Heo, T. J. Ham and J. W. Lee, "Unlocking Wordline-level Parallelism for Fast Inference on RRAM-based DNN Accelerator," 2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD), 2020, pp. 1-9.
- [7] H. Jiang, W. Li, S. Huang, S. Cosemans, F. Catthoor and S. Yu, "Analog-to-Digital Converter Design Exploration for Compute-in-Memory Accelerators," IEEE Design & Test, vol. 39, 2021
- [8] H.-S. P. Wong et al., "Metal–Oxide RRAM," in Proceedings of the IEEE, vol. 100, no. 6, pp. 1951-1970, June 2012, doi: 10.1109/JPROC.2012.2190369.
- [9] M. Hu et al., "Dot-product engine for neuromorphic computing: Programming 1T1M crossbar to accelerate matrix-vector multiplication," in DAC, 2016.

- [10]B. Feinberg, S. Wang and E. Ipek, "Making Memristive Neural Network Accelerators Reliable," in HPCA, 2018.
- [11]B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam and D. Kalenichenko, "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," in CVPR, 2018.
- [12]A. Hasso, K.Jacksi and K. Smith, "Effect of Quantization Error and SQNR on the ADC Using Truncating Method to the Nearest Integer Bit," in ICOASE, 2019
- [13]白欣怡, "記憶體內計算框架及逐層考量字線激活方法設計," 碩士論文, 中原大學資訊工程學系, 2023
- [14]A. Shahrestani, "Classifying CIFAR-10 using a simple CNN," <https://medium.com/analytics-vidhya/classifying-cifar-10-using-a-simple-cnn-4e9a6dd7600b>.
- [15]K. Simonyan, and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," arXiv:1409.1556.
- [16]M. -Y. Lin et al., "DL-RSIM: A Simulation Framework to Enable Reliable ReRAM-based Accelerators for Deep Learning," in ICCAD, 2018.
- [17]X. Peng, S. Huang, Y. Luo, X. Sun and S. Yu, "DNN+NeuroSim: An End-to-End Benchmarking Framework for Compute-in-Memory Accelerators with Versatile Device Technologies," in IEDM, 2019.
- [18]X. Peng, S. Huang, H. Jiang, A. Lu and S. Yu, "DNN+NeuroSim V2.0: An End-to-End Benchmarking Framework for Compute-in-Memory Accelerators for On-Chip Training," in IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 2021.
- [19]M. Imani, M. Samragh, Y. Kim, S. Gupta, F. Koushanfar and T. Rosing, "RAPIDNN: In-Memory Deep Neural Network Acceleration Framework," CoRR abs/1806.05794, 2018.
- [20]A. Ankit, I.E. Hajj, S.R. Chalamalasetti, G. Ndu, M. Foltin, R.S. Williams, P. Faraboschi, W. Hwu, J.P. Strachan, K. Roy and D.S. Milojicic, "PUMA: A Programmable Ultra-efficient Memristor-based Accelerator for Machine Learning Inference," CoRR abs/1901.10351 , 2019.